ECUAȚIA DE GRADUL AL II-LEA

Proiect Testarea Sistemelor Software
T1 Testare Unitară în Python

BILIUȚĂ Elisa-Adelina [11], MARCOVICI Anamaria [21] și
NEDELESCU George-Eduard [31]

[1]Universitatea din București/ Facultatea de Matematică și Informatică/
Programul de studii universitare de licență Matematică-Informatică

Anul III, Seria 31, Grupa 311

[1]elisa-adelina.biliuta@s.unibuc.ro
[2]anamaria.marcovici@s.unibuc.ro
[3]george-eduard.nedelescu@s.unibuc.ro

Cuprins

1 Prezentarea problemei **2**

2 Testare funcțională **4**

2.1 Partiționarea în clase de echivalență (CE) 4

2.2 Analiza valorilor de frontieră (AVF) 7

2.3 Partiționarea în categorii (PC) 9

Capitolul 1

Prezentarea problemei

În acest capitol prezentăm problema principală pe care o vom supune testării, anume rezolvarea unei ecuații de gradul al II-lea de forma $ax^2 + bx + c = 0$, cu $a \in \mathbb{R}^*$ și $b, c \in \mathbb{R}$.

Programul implementat primește trei coeficienți reali (a, b, și c) și calculează soluțiile ecuației pe baza discriminantului $\Delta = b^2 - 4ac$. De asemenea, programul tratează cazurile de excepție, cum ar fi validarea tipului de date introdus și restricția matematică pentru care coeficientul $a \neq 0$.

Distingem mai jos cele trei cazuri posibile, precum și formulele de calcul ale soluțiilor ecuației, în funcție de semnul lui Δ:

$$x_{1,2} = \begin{cases} \frac{-b \pm \sqrt{\Delta}}{2a}, & \text{dacă } \Delta > 0 \\ -\frac{b}{2a}, & \text{dacă } \Delta = 0 \\ \frac{-b \pm i\sqrt{|\Delta|}}{2a}, & \text{dacă } \Delta < 0 \end{cases}.$$

În continuare, prezentăm implementarea clasei `EquationSolver` în limbajul Python.

```python
import math

class EquationSolver:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def solve(self):
        if not all(isinstance(x, (int, float)) and not isinstance(x, bool) for x in [self.a, self.b, self.c]):
            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere reale!")

        if self.a == 0:
            raise ValueError("Coeficientul a nu poate fi 0!")

        delta = self.b ** 2 - 4 * self.a * self.c

        if delta < 0:
            real_part = -self.b / (2 * self.a)
            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
            x1 = complex(real_part, imag_part)
            x2 = complex(real_part, -imag_part)
            return x1, x2

        elif math.isclose(delta, 0.0, abs_tol=1e-9):
            return -self.b / (2 * self.a)

        else:
            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
            return x1, x2
```

Secvență de cod 1.1: Fișierul `equation_solver.py` ilustrând clasa `EquationSolver`

Capitolul 2

Testare funcțională

În acest capitol detaliem strategiile de testare funcțională de tip *Black Box* aplicate asupra clasei `EquationSolver`. Testarea funcțională presupune generarea datelor de test pe baza specificațiilor programului, fără a ține cont de structura internă a codului.

2.1 Partiționarea în clase de echivalență (CE)

Domeniul de intrări este format din cei trei coeficienți (a, b, c), care trebuie să fie numere reale, iar $a \neq 0$. Pe baza verificării tipului de date introduse, a restricțiilor și a calculului discriminantului (Δ), am identificat un domeniu de ieșiri format din 5 tipuri distincte de rezultate. În continuare, vom analiza fiecare dintre aceste domenii și clasele de echivalență generate.

Domeniul de intrări conține:

- un număr real nenul a: determină clasele $A_1 = \mathbb{R}^*$, $A_2 = \{0\}$ și $A_3 = \mathbb{R}^c$;
- un număr real b: determină clasele $B_1 = \mathbb{R}$ și $B_2 = \mathbb{R}^c$;
- un număr real c: determină clasele $C_1 = \mathbb{R}$ și $C_2 = \mathbb{R}^c$.

Observație 1. Am notat cu $\mathbb{R}^c$ complementara mulțimii numerelor reale. Dacă `x` $\in \mathbb{R}^c$, atunci `x` are tipul de dată diferit de `int` și `float` în reprezentarea internă a limbajului Python (și nu are nici tip `bool`), lucru ce va genera erori la validarea tipului de date.

Domeniul de ieșiri constă în rezultatele posibile returnate de metoda `solve()` și poate fi explicitat astfel:

- mulțimea soluțiilor ecuației: determină clasele $S_1 = \mathbb{C} \times \mathbb{C}$, $S_2 = \mathbb{R}$, $S_3 = \mathbb{R} \times \mathbb{R}$;
- stările de eroare: determină clasele: $E_1 =$ `ValueError` (excepție aruncată când cel puțin un coeficient aparține mulțimii $\mathbb{R}^c$) și $E_2 =$ `ValueError` (excepție aruncată pentru restricția matematică $a \neq 0$).

Observație 2. Clasa S_1 presupune un tuplu de două rădăcini complexe conjugate (când $\Delta < 0$), clasa S_2 presupune o singură rădăcină reală (rădăcină dublă, când $\Delta = 0$), iar clasa S_3 presupune un tuplu de două rădăcini reale distincte (când $\Delta > 0$).

Pe baza domeniului de ieșiri, folosind principiul metodei „reverse engineering” (de la ieșiri spre intrări) și ținând cont de fluxul de validare care previne evaluările multiple (o excepție aruncată oprește execuția), putem partiționa domeniul de intrări în trei clase disjuncte:

1. **Intrări care declanșează eroarea E_1 (Validarea tipului):** Execuția se oprește prematur dacă cel puțin un coeficient nu este număr real. Aceasta generează prima clasă de echivalență globală:

$$C_{E_1} = \{(a, b, c) \mid a \in A_3 \vee b \in B_2 \vee c \in C_2\}$$

2. **Intrări care nu declanșează eroarea E_1, dar care declanșează eroarea E_2 (Restricția matematică):** Acest caz presupune că toate datele sunt numere reale ($a \in A_1 \cup A_2$, $b \in B_1$, $c \in C_1$), dar coeficientul dominant este zero ($a \in A_2$). Obținem astfel a doua clasă de echivalență globală:

$$C_{E_2} = A_2 \times B_1 \times C_1 = \{(0, b, c) \mid b, c \in \mathbb{R}\}$$

3. **Intrări care nu declanșează nicio eroare:** Acest caz presupune date valide: avem coeficienți reali și $a \neq 0$, adică domeniul $A_1 \times B_1 \times C_1$. Facem discuție după calculul discriminantului Δ, care subpartiționează domeniul valid în cele 3 clase corespunzătoare soluțiilor:

$$C_{S_1} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac < 0\}$$
$$C_{S_2} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac = 0\}$$
$$C_{S_3} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac > 0\}$$

Pe baza partiționării anterioare, am selectat câte un caz de testare reprezentativ pentru fiecare clasă de echivalență. Seturile de date și rezultatele așteptate sunt sintetizate în tabelul de mai jos:

ID Test	Clasa	a	b	c	Δ	Rezultat așteptat
TC_CE_01	C_{E_1}	`"text"`	2	1	N/A	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_CE_02	C_{E_2}	0	2	1	N/A	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_CE_03	C_{S_1}	1	1	1	-3	$\boldsymbol{S_1}$ (Două rădăcini complexe)
TC_CE_04	C_{S_2}	1	2	1	0	$\boldsymbol{S_2}$ (Rădăcină reală dublă)
TC_CE_05	C_{S_3}	1	5	6	1	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)

Tabela 2.1: Cazuri de testare pentru metoda CE

Observație 3. Se remarcă faptul că pentru clasele de excepție C_{E_1} și C_{E_2}, discriminantul Δ nu este calculat, deoarece execuția este întreruptă înainte de a ajunge la evaluarea formulei matematice. Această afirmație nu încalcă principiile strategiei *Black Box*.

Pentru a valida practic scenariile teoretice definite anterior, în secvența de cod următoare prezentăm implementarea automatizată a cazurilor de testare extrase din Tabelul 2.1, utilizând frameworkul `unittest` din limbajul Python. Deoarece urmează se efectuează calcule analitice pe valori reale, verificările folosesc extensiv mecanismul `assertAlmostEqual` pentru a preveni eșuarea testelor pe fondul erorilor inevitabile de trunchiere și rotunjire în virgulă mobilă.

```python
class TestEquationSolverCE(unittest.TestCase):
    # TC_CE_01: Clasa CE1 - Validarea tipului de date (cel putin un coeficient nu este numar real); vom trata fenomenul defect masking
    def test_tc_ce_01_a_invalid(self):
        solver = EquationSolver("text", 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    def test_tc_ce_01_b_invalid(self):
        solver = EquationSolver(1, "text", 1)
        with self.assertRaises(ValueError):
            solver.solve()

    def test_tc_ce_01_c_invalid(self):
        solver = EquationSolver(1, 2, "text")
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_CE_02: Clasa CE2 - Restrictia matematica (a != 0)
    def test_tc_ce_02_a_zero(self):
        solver = EquationSolver(0, 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_CE_03: Clasa CS1 - Delta < 0 (Doua radacini complexe conjugate)
    def test_tc_ce_03_delta_negativ(self):
        solver = EquationSolver(1, 1, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.5)
        self.assertAlmostEqual(x2.real, -0.5)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)

    # TC_CE_04: Clasa CS2 - Delta = 0 (O radacina reala dubla)
    def test_tc_ce_04_delta_zero(self):
        solver = EquationSolver(1, 2, 1)
        x = solver.solve()

        self.assertIsInstance(x, float)
        self.assertAlmostEqual(x, -1.0)

    # TC_CE_05: Clasa CS3 - Delta > 0 (Doua radacini reale distincte)
    def test_tc_ce_05_delta_pozitiv(self):
        solver = EquationSolver(1, 5, 6)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
        self.assertAlmostEqual(min(x1, x2), -3.0, places=5)
```

Secvență de cod 2.1: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverCE`

2.2 Analiza valorilor de frontieră (AVF)

Conform principiilor de bază ale metodei AVF, pentru fiecare frontieră identificată trebuie testată atât valoarea exactă a limitei, cât și valorile imediat adiacente (din interiorul și din afara clasei valide). Pentru problema ecuației de gradul al II-lea, deși prelucrăm un domeniu continuu, reprezentat de mulțime $\mathbb{R}$, distingem clar două frontiere decizionale impuse de fluxul de control al programului: trecerea dintre valid și invalid ($a = 0$) și punctul critic dintre rădăcinile reale și cele complexe ($\Delta = 0$). Să detaliem:

- **Frontiera coeficientului dominant (a):** Restricția matematică impune $a \neq 0$. Conform teoriei AVF, setul de teste trebuie să evalueze cazul-limită ($a = 0$) și valorile imediat adiacente acestuia ($a \to 0^+$ și $a \to 0^-$). Întrucât testul pentru valoarea $a = 0$ a fost deja executat sub umbrela metodei CE (reprezentând clasa de excepție C_{E_2}), analiza de frontieră va fi completată prin injectarea unor valori arbitrar de mici, notate cu $\pm\epsilon$ (unde vom alege $\epsilon = 0.001$).
- **Frontiera discriminantului (Δ):** Valoare critică care produce separarea rădăcinilor reale de cele complexe este $\Delta = 0$. Cazul-limită ($\Delta = 0$) reprezintă însăși clasa C_{S_2} din metoda CE. Tehnica AVF impune completarea cu testarea vecinătăților $\Delta \to 0^+$ și $\Delta \to 0^-$, stări care pot fi forțate prin variația fină a coeficientului b în jurul valorii 2, menținând $a = 1$ și $c = 1$, de exemplu.

Pentru a oferi o imagine completă a acoperirii la frontieră, Tabelul 2.2 sintetizează întregul spectru de teste AVF, înglobând atât valorile pentru frontieră (moștenite din partiționarea de echivalență), cât și cazurile pentru vecinătățile acesteia.

ID Test	Frontieră	a	b	c	Δ	Rezultat așteptat
TC_CE_02	$a = 0$	0	2	1	N/A	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_AVF_01	$a \to 0^-$	-0.001	2	1	4.004	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)
TC_AVF_02	$a \to 0^+$	0.001	2	1	3.996	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)
TC_CE_04	$\Delta = 0$	1	2	1	0	$\boldsymbol{S_2}$ (Rădăcină reală dublă)
TC_AVF_03	$\Delta \to 0^-$	1	1.999	1	≈ -0.004	$\boldsymbol{S_1}$ (Două rădăcini complexe)
TC_AVF_04	$\Delta \to 0^+$	1	2.001	1	≈ 0.004	$\boldsymbol{S_3}$ (Două rădăcini reale distincte)

Tabela 2.2: Cazuri de testare pentru metoda AVF

Observație 4. Spre deosebire de mulțimile de tip `int`, unde valoarea adiacentă este precis determinabilă ($+1$ sau -1), pe un domeniu continuu de tip `float` conceptul de „valoare imediat adiacentă” este mărginit doar de limitele de reprezentare arhitecturală a mașinii. Din perspectiva strategiei *Black Box*, o toleranță mică $\epsilon = 10^{-3}$ este optimă și suficientă pentru a garanta evaluarea ramurilor decizionale la extremitățile lor critice.

Implementarea automatizată a scenariilor AVF este redată mai jos.

```python
class TestEquationSolverAVF(unittest.TestCase):
    # Testele pentru a=0 si Delta=0 au fost deja implementate in clasa TestEquationSolverCE (test_tc_ce_02 si test_tc_ce_04)

    # TC_AVF_01: Limita stanga pentru coeficientul a (a -> 0-)
    def test_tc_avf_01_a_limita_negativa(self):
        solver = EquationSolver(-0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
        self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)

    # TC_AVF_02: Limita dreapta pentru coeficientul a (a -> 0+)
    def test_tc_avf_02_a_limita_pozitiva(self):
        solver = EquationSolver(0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
        self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)

    # TC_AVF_03: Limita stanga pentru Delta (Delta -> 0-)
    def test_tc_avf_03_delta_limita_negativa(self):
        solver = EquationSolver(1, 1.999, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.9995, places=3)
        self.assertAlmostEqual(x2.real, -0.9995, places=3)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)

    # TC_AVF_04: Limita dreapta pentru Delta (Delta -> 0+)
    def test_tc_avf_04_delta_limita_pozitiva(self):
        solver = EquationSolver(1, 2.001, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
        self.assertAlmostEqual(min(x1, x2), -1.03213, places=1)
```

Secvență de cod 2.2: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverAVF`

2.3 Partiționarea în categorii (PC)

Metoda partiționării în categorii extinde tehnicile anterioare, având ca scop generarea unui set de date de test care acoperă exhaustiv funcționalitatea sistemului și maximizează posibilitatea descoperirii erorilor. Această abordare structurată descompune specificația în unități testabile, identifică parametrii, categoriile acestora și partiționează fiecare categorie în alternative. Pentru a preveni explozia combinatorică inerentă și a elimina cazurile care nu au sens, se introduc constrângeri logice (condiții de selecție) asupra alternativelor.

Urmărind pașii metodologici, aplicăm metoda PC pentru clasa `EquationSolver`:

Pasul 1: Descompunerea specificației în unități

Sistemul prezintă o singură unitate funcțională testabilă izolată: metoda `solve()`.

Pasul 2: Identificarea parametrilor și a condițiilor de mediu

Parametrii de intrare care influențează direct comportamentul sunt coeficienții a, b, c. Condiția de mediu relevantă (starea internă rezultată în urma procesării datelor valide) este discriminantul Δ.

Pasul 3: Găsirea categoriilor

Proprietățile majore care definesc și separă fluxurile de execuție ale programului sunt:

- $\boldsymbol{C_1}$ *(Validitatea tipului de date)*: verifică apartenența $a, b, c \in \mathbb{R}$;
- $\boldsymbol{C_2}$ *(Valoarea coeficientului dominant)*: impune restricția $a \neq 0$;
- $\boldsymbol{C_3}$ *(Natura rădăcinilor)*: dictată de semnul lui Δ.

Pașii 4 și 5: Partiționarea în alternative și scrierea specificației de testare

Pentru a eficientiza testarea și a evita generarea de combinații redundante, asociem alternativelor constrângeri. Proprietatea `[error]` va opri combinarea ulterioară, deoarece declanșează întreruperea execuției programului. Pentru a asigura o testare robustă, alternativele integrează și valorile de frontieră identificate anterior. Să detaliem:

- **Parametrul a:**
  1. $a \in \mathbb{R}^c$ `[error]`;
  2. $a = 0$ `[error]`;
  3. $a \to 0^-$ `[property Valid_A]`;
  4. $a \to 0^+$ `[property Valid_A]`;
  5. $a \in \mathbb{R} \setminus [-\epsilon, \epsilon]$ `[property Valid_A]`.
- **Parametrul b:**
  1. $b \in \mathbb{R}^c$ `[error]`;
  2. $b \in \mathbb{R}$ `[property Valid_B]`.
- **Parametrul c:**
  1. $c \in \mathbb{R}^c$ `[error]`;
  2. $c \in \mathbb{R}$ `[property Valid_C]`.
- **Semnul lui Δ:**
  1. $\Delta \ll 0$ `[if Valid_A and Valid_B and Valid_C]`;
  2. $\Delta \to 0^-$ `[if Valid_A and Valid_B and Valid_C]`;
  3. $\Delta = 0$ `[if Valid_A and Valid_B and Valid_C]`;
  4. $\Delta \to 0^+$ `[if Valid_A and Valid_B and Valid_C]`;
  5. $\Delta \gg 0$ `[if Valid_A and Valid_B and Valid_C]`.

Pașii 6 și 7: Crearea cazurilor și a datelor de test

În absența constrângerilor, simplul produs cartezian al alternativelor ar fi generat $5 \times 2 \times 2 \times 5 = 100$ de combinații. Aplicând condițiile de selecție, excludem evaluarea lui Δ pentru intrările care declanșează erori. De asemenea, pentru optimizare, într-un caz de testare la limita frontierelor, testăm doar câte o singură frontieră, reducând astfel spațiul teoretic la 11 scenarii de testare distincte, care pot fi observate în Tabelul 2.3.

Acest set de teste este optim, deoarece cuprinde:

- **4 cazuri de excepție**: acoperă validarea datelor (3 teste) și restricția matematică esențială (1 test);
- **2 cazuri pentru frontiera intrărilor**: testează stabilitatea programului când a tinde la 0 din ambele direcții;
- **2 cazuri pentru frontiera stărilor interne**: testează sensibilitatea decizională la granița dintre soluțiile reale și cele complexe;
- **3 cazuri de clasă normală**: validează comportamentul standard pentru cele trei tipuri de rădăcini.

ID Test	Combinații	a	b	c	Rezultat așteptat
TC_PC_01	a_1	True	2	1	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_PC_02	$a_5 \wedge b_1$	1	None	1	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_PC_03	$a_5 \wedge b_2 \wedge c_1$	1	2	[]	$\boldsymbol{E_1}$ (Excepție: tipuri invalide)
TC_PC_04	$a_2 \wedge b_2 \wedge c_2$	0	2	1	$\boldsymbol{E_2}$ (Excepție: a nu poate fi 0)
TC_PC_05	$a_3 \wedge b_2 \wedge c_2 \wedge \Delta_5$	-0.001	2	1	$\boldsymbol{S_3}$ ($a \to 0^-$; $\Delta > 0$)
TC_PC_06	$a_4 \wedge b_2 \wedge c_2 \wedge \Delta_5$	0.001	2	1	$\boldsymbol{S_3}$ ($a \to 0^+$; $\Delta > 0$)
TC_PC_07	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_2$	1	1.999	1	$\boldsymbol{S_1}$ (a normal; $\Delta \to 0^-$)
TC_PC_08	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_3$	1	2	1	$\boldsymbol{S_2}$ (a normal; $\Delta = 0$)
TC_PC_09	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_4$	1	2.001	1	$\boldsymbol{S_3}$ (a normal; $\Delta \to 0^+$)
TC_PC_10	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_1$	1	1	1	$\boldsymbol{S_1}$ (valori normale; $\Delta < 0$)
TC_PC_11	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_5$	1	5	6	$\boldsymbol{S_3}$ (valori normale; $\Delta > 0$)

Tabela 2.3: Cazuri de testare pentru metoda PC

Pentru a încheia secțiunea, prezentăm implementarea automatizată a scenariilor sumarizate mai sus în fragmentul de cod următor.

```python
class TestEquationSolverPC(unittest.TestCase):
    # TC_PC_01 -> TC_PC_03: Erori de validare a tipului de date pentru coeficientii a, b, c
    def test_tc_pc_01_a_invalid(self):
        solver = EquationSolver(True, 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    def test_tc_pc_02_b_invalid(self):
        solver = EquationSolver(1, None, 1)
        with self.assertRaises(ValueError):
            solver.solve()

```

```python
    def test_tc_pc_03_c_invalid(self):
        solver = EquationSolver(1, 2, [])
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_PC_04: Valoare invalida matematic (a = 0)
    def test_tc_pc_04_a_zero(self):
        solver = EquationSolver(0, 2, 1)
        with self.assertRaises(ValueError):
            solver.solve()

    # TC_PC_05: Limita stanga pentru coeficientul a (a -> 0-)
    def test_tc_pc_05_a_limita_negativa(self):
        solver = EquationSolver(-0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
        self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)

    # TC_PC_06: Limita dreapta pentru coeficientul a (a -> 0+)
    def test_tc_pc_06_a_limita_pozitiva(self):
        solver = EquationSolver(0.001, 2, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
        self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)

    # TC_PC_07: Limita stanga pentru Delta (Delta -> 0-)
    def test_tc_pc_07_delta_limita_negativa(self):
        solver = EquationSolver(1, 1.999, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.9995, places=3)
        self.assertAlmostEqual(x2.real, -0.9995, places=3)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)

    # TC_PC_08: Delta = 0 (O radacina reala dubla)
    def test_tc_pc_08_delta_zero(self):
        solver = EquationSolver(1, 2, 1)
        x = solver.solve()

        self.assertIsInstance(x, float)
        self.assertEqual(x, -1.0)

    # TC_PC_09: Limita dreapta pentru Delta (Delta -> 0+)
    def test_tc_pc_09_delta_limita_pozitiva(self):
        solver = EquationSolver(1, 2.001, 1)
```

```python
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
        self.assertAlmostEqual(min(x1, x2), -1.03213, places=3)

    # TC_PC_10: Delta < 0 (Doua radacini complexe)
    def test_tc_pc_10_delta_negativ(self):
        solver = EquationSolver(1, 1, 1)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, complex)
        self.assertIsInstance(x2, complex)

        self.assertAlmostEqual(x1.real, -0.5)
        self.assertAlmostEqual(x2.real, -0.5)

        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)

    # TC_PC_11: Delta > 0 (Doua radacini reale distincte)
    def test_tc_pc_11_delta_pozitiv(self):
        solver = EquationSolver(1, 5, 6)
        x1, x2 = solver.solve()

        self.assertIsInstance(x1, float)
        self.assertIsInstance(x2, float)

        self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
        self.assertAlmostEqual(min(x1, x2), -3.0, places=5)
```

Secvență de cod 2.3: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverPC`

Rezultatul obținut la rularea tuturor testelor din acest capitol este prezentat mai jos:

```
"D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\.venv\Scripts\
    python.exe" "D:/SOFTURI/PyCharm 2025.3.3/plugins/python-ce/helpers/
    pycharm/_jb_unittest_runner.py" --path "D:\FMI\ANUL III\TESTAREA
    SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py"
Testing started at 17:02 ...
Launching unittests with arguments python -m unittest D:\FMI\ANUL III\
    TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py --quiet
     in D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS

Ran 22 tests in 0.024s

OK

Process finished with exit code 0
```